MODERN CRYPTOGRAPHY MINI PROJECT

Lightweight Feistel-Based Smart Meter Encryption Suite

Project scope, system design, evaluation plan and team responsibilities

Domain: Symmetric-key cryptography and smart-grid security

Implementation: Python, FastAPI and web-based visualization

Team size: Five members

1. Project Overview

The project develops a simulation of a residential smart meter that encrypts electricity-usage readings before sending them to a utility provider. Its core is a custom 8-bit Feistel cipher derived from the structure of DES. The reduced block size makes each operation easy to observe and suitable for studying resource-constrained devices.

The cipher is an educational model, not a replacement for AES in a real billing system. The study focuses on the trade-off between computational cost and cryptographic diffusion as the number of rounds changes.

2. Objectives

- Implement correct encryption and decryption for 8-bit data blocks.
- Generate realistic dummy household electricity readings.
- Visualize every Feistel round through a web interface.
- Compare 4-, 6- and 8-round configurations.
- Measure execution time, ciphertext diffusion and estimated energy use.
- Demonstrate tampering and replay protection in a simulated meter packet.

3. System Flow

> **Meter data generation → 8-bit quantization → Feistel encryption → packet transmission → validation → decryption → utility dashboard**

A reading contains a meter ID, timestamp, sequence number and energy value. The energy value is mapped to an integer from 0 to 255 before encryption. The original and quantized values are both retained so that any loss of precision is visible.

4. Cipher Design

An 8-bit plaintext block is divided into two 4-bit halves, *L* and *R*. Each round uses a 4-bit subkey derived from a 16-bit master key. The round function applies XOR, substitution and bit permutation.

```
F(R, Kᵢ) = Permute(SBox(R XOR Kᵢ))
L(new) = R
R(new) = L XOR F(R, Kᵢ)
```

Decryption applies the round keys in reverse order. The final-swap and output-format convention will be fixed in the algorithm specification and verified using known-answer tests.

Property	DES	Project Cipher
Block size	64 bits	8 bits
Half size	32 bits	4 bits
Effective key size	56 bits	16-bit baseline
Rounds	16	4, 6 or 8
Purpose	General-purpose encryption	Educational lightweight simulation

5. Dummy Meter Data

The generator will produce a time series with a small base load, morning and evening peaks, occasional appliance spikes and measurement noise. Each generated report will use the following structure:

```
meter_id | timestamp | sequence_number | usage_kwh | quantized_value
```

6. Web Application

FastAPI will provide the backend API and keep the cryptographic logic separate from the user interface. The frontend will use HTML, CSS and JavaScript, with Chart.js or Plotly for graphs.

Module	Main functions
Live Meter	Current reading, time-series graph and simulation controls
Cipher Laboratory	Binary input, round keys, S-box output and round-by-round state
Communication	Packet contents, sequence validation, tampering and replay demo
Security Analysis	Avalanche tests, key sensitivity and diffusion heatmap
Performance	Timing, estimated energy and round-count comparison

7. Evaluation

Execution time

A large batch of blocks will be timed to reduce measurement noise. Mean time per block, variation and throughput will be reported separately for 4, 6 and 8 rounds. Web request latency will not be included in cipher timing.

Ciphertext diffusion

One plaintext bit will be changed and both inputs will be encrypted using the same key. The number of changed ciphertext bits gives the avalanche score. Tests will cover every input-bit position across many plaintext and key samples.

```
Avalanche (%) = changed ciphertext bits / 8 × 100
```

Power consumption

The initial web version will estimate energy from an identified microcontroller's voltage, current and execution time. If hardware is available, the cipher will also be implemented in C and tested using Wokwi or an Arduino with a current sensor.

```
Energy = Voltage × Current × Execution Time
```

8. Team Responsibilities

Member	Responsibility	Deliverables
1	Cipher design	Feistel algorithm, key schedule, S-box, encryption/decryption and unit tests
2	Meter simulation	Dummy readings, quantization, packets, timestamps and replay simulation
3	Backend	FastAPI endpoints, input validation, integration and API tests
4	Frontend	Dashboard, round visualization, graphs and responsive layout

Member	Responsibility	Deliverables
5	Evaluation	Avalanche analysis, benchmarks, energy model and comparison charts

Each member will maintain tests and documentation for their module, contribute through version control, and demonstrate their work during the final presentation.

9. Development Schedule

Stage	Work	Output
1	Research and algorithm specification	Final design and hand-worked example
2	Python cipher and automated tests	Verified reference implementation
3	Meter and packet simulation	End-to-end data flow
4	Evaluation scripts	Timing, diffusion and energy results
5	Web interface and integration	Working demonstration
6	Testing, report and presentation	Final submission

10. Expected Result and Limitation

The study is expected to show that additional rounds improve diffusion while increasing time and energy cost. The 8-bit design is useful for demonstrating Feistel construction, but its small block and key spaces make it unsuitable for production smart-meter security. A real deployment should use a standard authenticated cipher such as AES-based encryption.